Explainability in Reinforcement Learning

Maxime Alaarabiou Nicolas Delestre Laurent Vercouter

May 14, 2026

Chapter 1

Deep Learning

This chapter introduces the fundamental concepts of Deep Learning (DL), providing the necessary foundation for the ideas developed throughout the rest of this manuscript. This chapter can be seen as a synthesis of the reference work Deep Learning [3]. Its structure is as follows:

- A overview of the historical developments that led to the emergence of the field.
- A formalization of the objective underlying DL algorithms.
- An examination of the structure of artificial Neural Network (NN).
- An explanation of how the learning process shapes NN.
- A discussion of how such models are evaluated.

1.1 Historical Developments

DL and NN emerged from an ambition to study the expressive capabilities of organic brain. They can be seen as a particularly striking example of biomimicry, as they draw inspiration from living systems to address scientific challenges. The first major milestone came in 1943 with the proposal of a mathematical model of an artificial neuron, with a formulation of networked interactions [8]. This work established the key idea that networks of simple units can give rise to complex computations. The focus was on representational power, not on the ability of such systems to learn. As a result, this work remained largely theoretical and had limited practical applications. A major step toward operationalizing the idea of networks with the perceptron algorithm in 1958 [11]. The perceptron was designed to adjusting its parameters based on inputoutput pairs in order to approximate a function. However, the perceptrons fundamental limitations were exposed in 1969, revealing that the algorithm could not learn functions as simple as the exclusive-or [9]. To overcome this limitation, researchers in 1986 proposed using gradient descent optimization to train deep NNs [12].

It took several decades before DL reached the level of prominence it has today. One major obstacle was the absence of theoretical guarantees regarding the convergence of learning algorithms. Although the universal approximation

theorem shows that NN can represent a wide class of functions [5], it did not ensure that such representations could be effectively learned through optimization methods like gradient descent. This lack of rigorous guarantees initially reduced interest within the academic community. As a result, early progress relied largely on empirical breakthroughs.

These empirical advances were made possible by two key developments: the availability of large-scale datasets and significant improvements in computational infrastructure. DL methods require vast amounts of data to perform well, and the rise of the internet alongside the digitization of information enabled the creation of such datasets [4]. At the same time, progress in hardware, particularly the use of graphics processing units, greatly increased computational capacity and made it feasible to train large-scale models [7].

These societal changes paved the way for the rise of DL as we know it today. DL has achieved remarkable success across a wide range of domains, including computer vision [7], complex games [13], content recommendation [2], bio-chemistry [6], as well as the now essential digital assistants based on large language models [1]. The diversity of these applications explains the enthusiasm for this technology, which continues to redefine the boundaries of what was once considered automatable.

1.2 Objective

Now that we have outlined the main developments that led to the rise of DL, we need to formalize what we are actually trying to achieve when performing DL. The range of problems that can be addressed with DL is extremely broad. To ensure clarity, we will consider the supervised learning setting, specifically applied to a regression task. This framework serves as a standard reference in the field, as all building blocks of DL applications are built upon this fundamental formalism.

In this setting, learning consists of progressively shaping a NN that serves as an approximation of a target function. At initialization, the network does not yet provide a meaningful representation of this function. It gradually becomes capable of capturing the relationship between inputs and outputs through successive updates during training. For this reason, DL is naturally situated within the framework of machine learning. A standard definition of machine learning is:

> "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E." [10]

When this formulation is adapted to DL, we obtain:

- The computer program corresponds to the learning algorithm, which is responsible for adjusting the network's parameters.
- The task T consists of learning a NN that provides a good approximation of the target function.
- The performance measure P corresponds to a function used to evaluate the quality of this approximation.

- The experience E takes the form of a dataset composed of input–output pairs derived from the function to be approximated.

We now introduce the mathematical formalism used in this chapter in order to remove any ambiguity in the developments that follow. The function we aim to approximate is denoted by f. As with any function, it defines a mapping from an input to an output: we write $x \in \mathcal{X}$ for an input and $y \in \mathcal{Y}$ for the corresponding output. The function f is therefore characterized by an input space $\mathcal{X}$ and an output space $\mathcal{Y}$, and can be formally written as $f : \mathcal{X} \to \mathcal{Y}$. For simplicity of exposition and notation, we will assume in what follows that both $\mathcal{X}$ and $\mathcal{Y}$ are vector spaces, while keeping in mind that this framework can be readily generalized to more settings. Throughout this manuscript, the notation $\hat{\cdot}$ will be used to indicate that a given object is an approximation of another. Since the goal of the NN we train is to approximate f, we denote this approximation by $\hat{f} : \mathcal{X} \to \mathcal{Y}$. Given an input $x \in \mathcal{X}$, the output of the approximating function $\hat{f}(x)$ is denoted by $\hat{y}$.

It is important to emphasize that the target function f is generally not known explicitly. If it were, there would be no need to learn an approximation, since the true function would already provide the exact mapping. In practice, we only have access to a finite set of observations generated by f, organized in what is called a dataset. To distinguish between the different samples, we introduce an index i, allowing us to define input–output pairs of the form $(x^{(i)}, y^{(i)})$. This leads to the following formulation for a dataset $\mathcal{D}$ of size N, where N denotes the number of available examples:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}.$$

Now that we have made explicit how the target function f is represented, we can ask what it means for a function $\hat{f}$ to be a good approximation of this function. It means that, for a given input $x^{(i)}$, the NN output $\hat{y}^{(i)}$ is close to the true output $y^{(i)}$. To quantify the quality of this approximation, we introduce a function $\mathcal{L}$ that measures the discrepancy between predictions and target values. In DL, this function called the loss function plays a central role as it guides the learning process. In regression settings, a common choice is the mean squared error, defined as:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2.$$

Thanks to this loss, it is now possible to rigorously define the performance measure P. This performance measure corresponds to the average loss over the dataset and can therefore be written as:

$$P = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{f}(x^{(i)}), y^{(i)}).$$

This formulation captures the mathematical core of the learning objective. We seek a NN $\hat{f}$ that minimizes the prediction error over the dataset $\mathcal{D}$. As you can see, we are working in a very general setting, where the target function f may take many different forms. Consequently, the network $\hat{f}$ must be expressive enough to approximate a wide variety of such functions. This requirement motivates the choice of NN architectures, which are specifically designed to represent highly diverse functional mappings.

1.3 Neural Network Structure

NN are called connectionist models. Rather than modeling information processing through a set of explicit rules, they rely on a flow of signals distributed within a computational structure. Their architecture is composed of a large number of simple computational units, which operate on partial representations and exchange signals across the network. A complex pattern of weighted connections between these units enables the transformation and propagation of information.

To simplify the presentation throughout this chapter, we focus on the Multi-Layer Perceptron (MLP) architecture. Although more specialized architectures exist, the MLP forms a fundamental building block of modern DL. Indeed, most of the essential mechanisms of DL are present in it, making it a particularly suitable framework for introducing key concepts.

1.3.1 Neurons

The artificial neuron is the basic building block of NN. It is through these units that all computations within the network are carried out. A neuron receives a set of input signals from other neurons and produces an output that is transmitted to subsequent neurons units. Its internal state, called the activation, reflects its level of response and encodes the information processed for a given input. This activation, or lack thereof, is then used as input information by other neurons to determine their own activations, thereby propagating information through the network.

A neuron is characterized by:

- a set of incoming connections from other neurons,
- a weight associated with each of these connections,
- a bias term,
- a differentiable nonlinear activation function,
- a scalar activation state.

The activation of a neuron is determined by a weighted combination of its input signals, to which a bias is added, followed by the application of a nonlinear activation function. Mathematically, for a neuron receiving inputs $(a_1, \ldots, a_n)$, its activation a is given by:

$$a = \sigma\left(\sum_{i=1}^{n} w_i a_i + b\right),$$

where:

- w_i is the weight associated with the i-th incoming connection,
- b is the bias,
- σ is the activation function.

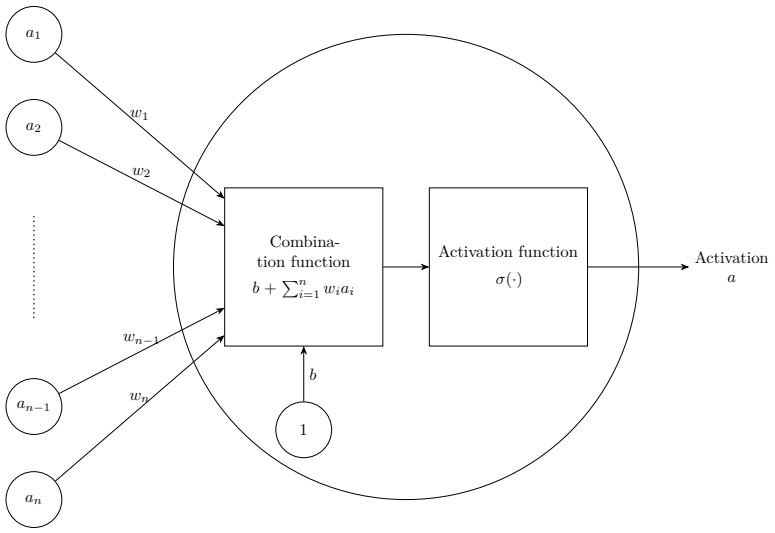

Figure 1.1: Artificiale neurone.

Figure 1.1 shows a schematic view of this basic unit. The set of weights $w_1, \ldots, w_i$ with the bias b constitute the parameters of the neuron, and they are the elements that evolve during training. The magnitude of a weight w (relative to the others) indicates how strongly a neuron depends on the activation of the neurons to which it is connected. A large positive weight means that a high activation in the connected neuron tends to increase the activation of the receiving neuron, whereas a negative weight implies the opposite effect. In this way, the weights modulate the strength and nature of the connections between neurons, thereby defining the overall pattern of interaction within the network.

The activation function σ models how a neuron transforms incoming signals into its output activation. Originally, this function was inspired by biological neurons, with the goal of mimicking the firing behavior of organic cells. However, modern DL no longer relies on this biological interpretation. In practice, any function that is nonlinear, differentiable, and computationally efficient can be used. Nonlinearity is a crucial property, since without it a NN would reduce to a composition of linear transformations, and would therefore be unable to approximate complex functions.

1.3.2 Layers

In MLP architectures, neurons are organized into successive layers, as illustrated in Figure 1.2. This layered structure determines how neurons are connected. In this setting, each neuron receives inputs from all neurons in the preceding layer and sends its output to all neurons in the following layer. Consequently, neurons within the same layer do not interact with each other.

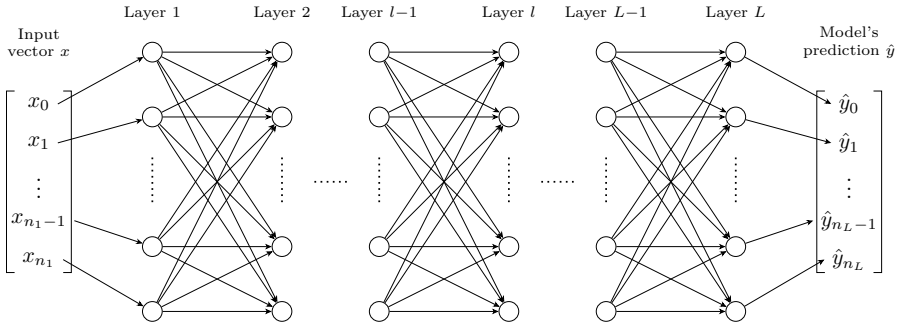

Figure 1.2: Multilayer Perceptron Architecture.

Three types of layers are distinguished:

- an input layer, which directly receives input,
- one or more hidden layers, which perform intermediate transformations,
- an output layer, which produces the final prediction.

The input and output layers play a specific role. The input layer has no preceding layer: its neurons do not compute activations but instead directly encode the components of the input vector x. Conversely, the output layer has no subsequent layer, and its neurons produce the model's prediction $\hat{y}$. This structure imposes a direct correspondence between the dimensionality of the input space and the number of neurons in the input layer, and similarly between the output space and the number of neurons in the output layer.

When describing NN, especially in MLP, it is more common to reason at the level of layers rather than individual neurons. This perspective allows interactions between layers to be modeled in a matrix form. Such a representation enables efficient parallel computation and forms the basis of modern DL implementations [7]. Within this framework, it is useful to introduce the notion of layer activations. The activation of a layer corresponds to the activations of all its neurons, organized into a vector. For a layer l composed of n_l neurons, the activations are given by:

$$\mathbf{h}^{(l)} = [a_1^{(l)}, \ldots, a_{n_l}^{(l)}].$$

Using this formulation, the interaction between two consecutive layers can be written compactly as:

$$\mathbf{h}^{(l)} = \sigma^{(l)} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right),$$

where:

- $\mathbf{h}^{(l-1)}$ represents the activations vector of the previous layer,
- $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix connecting layer $l-1$ to layer l,
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the bias vector associated with layer l,
- $\sigma^{(l)}$ is the activation function applied element-wise to the neurons of layer l.

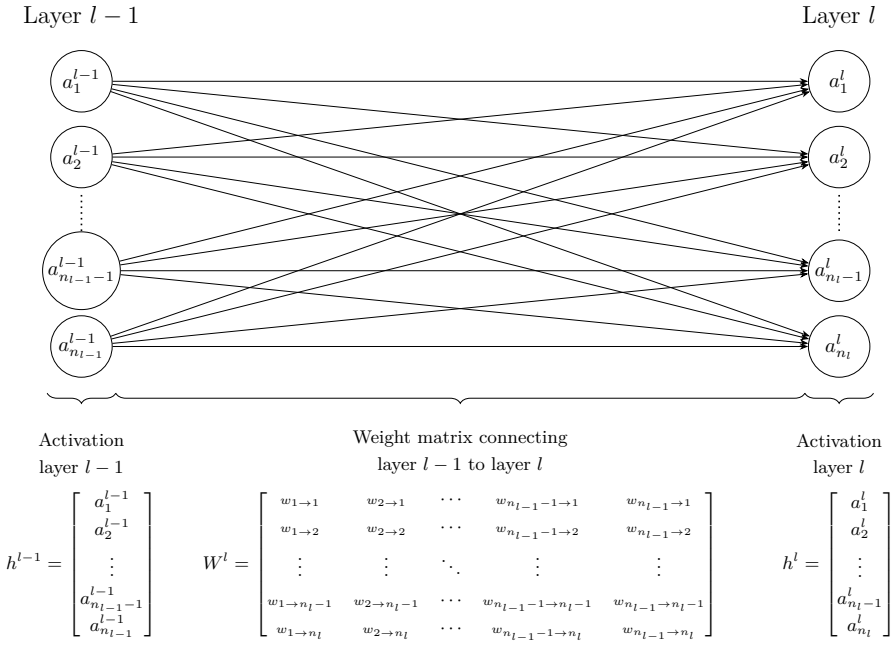

Figure 1.3: Interaction between layers.

This matrix-vector representation is visualised in Figure 1.3, which highlights how each neuron in layer l receives weighted signals from all neurons in the previous layer.

It is important to examine the role played by the intermediate layers of a NN. At least one hidden layer is required to represent sufficiently complex functions. In practice, however, modern architectures often include many more. This depth is mainly motivated by empirical observations. Each layer is seen as a processing step, and more complex target functions typically require more such steps to be well approximated. However, this interpretation is based on experimental findings rather than a theoretical justification [3]. An entire chapter of this manuscript is devoted to studying the information that can be extracted from the activations of these intermediate layers.

1.3.3 Network

With a layer-wise representation, the propagation of information can be described as a sequence of transformations applied from the input layer (indexed 1) to the output layer (indexed L):

$$\hat{f}(x) = \mathbf{h}^{(L)} = \sigma^{(L)}\left(\mathbf{W}^{(L)}\sigma^{(L-1)}\left(\dots\sigma^{(1)}\left(\mathbf{W}^{(1)}x + \mathbf{b}^{(1)}\right)\dots\right) + \mathbf{b}^{(L)}\right) = \hat{y}.$$

For a more compact description of information propagation, the layer activations are defined recursively as:

$$\mathbf{h}^{(l)} = \begin{cases} x & \text{if } l = 1, \\ \sigma^{(l)}\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) & \text{if } 2 \le l \le L. \end{cases}$$

To simplify notation, it is convenient to group all parameters of the network into a single set. The model parameters are thus denoted by:

$$\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}.$$

In this manuscript, we consider θ as a vector in $\mathbb{R}^P$, where P denotes the total number of parameters $\theta = [w_1, \ldots, w_P]$. The notation $\hat{f}_\theta$ expresses the dependence of the NN on its parameters. Increasing the number of parameters, either by adding more neurons per layer or by increasing the number of layers, enhances the expressive power of the model, allowing it to approximate more complex functions. However, this also increases computational cost and training time.

It is important to distinguish between the architecture and the parameters of a NN. When we refer to the architecture of $\hat{f}$, we mean the number of layers, the number of neurons per layer, and the choice of activation functions. The parameters correspond to θ, which controls the strength of the connections between neurons.

1.4 Learning

Now that we have detailed the structure of a NN, we must turn our attention to how it is modified in order to accomplish the task assigned to it. To do so, we need to focus on what lies at the very core of DL: the learning process itself. As a reminder, the objective of a NN is to adjust its parameters so that it becomes a good approximation of a target function. That is, for a given architecture $\hat{f}$, the goal is to find the set of parameters θ that best approximates the target function f. Mathematically, this objective is expressed as:

$$\min_\theta \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}).$$

However, the NN $\hat{f}$ used to approximate the target function f is highly complex. It involves a large number of parameters as well as many nonlinear components. As a result, we cannot find an exact solution by directly studying the properties of the function. So, one must therefore rely on approximate methods to optimize NNs. The class of algorithms used for this optimization is known as gradient descent methods.

For simplicity, we focus here on Stochastic Gradient Descent (SGD) with a batch size of 1. As in previous cases, this choice is made because this setting captures the essential intuition behind the training method.

The first step of SGD is to randomly initialize the parameter vector $\theta \in \mathbb{R}^P$ for a given network architecture $\hat{f}$. We denote this initial state by $\theta^{(1)}$. The goal is then to iteratively modify this vector in order to improve the network's ability to approximate the target function.

To determine how to update the parameters, we study the effect of small variations in each parameter on the loss $\mathcal{L}$. To analyze how parameter variations affect the loss, we rely on a fundamental mathematical tool known as the gradient. It is defined as the collection of partial derivatives of a function with respect to its variables. In our setting, the objective is to understand, for a given example

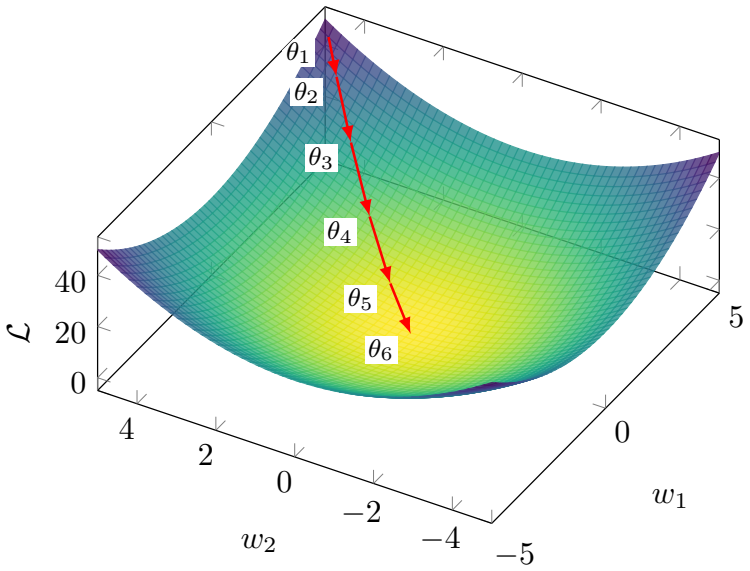

Figure 1.4: Gradient descent in a case $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$.

$(x^{(i)}, y^{(i)})$, how the parameters should be adjusted so that the NN prediction $\hat{y}^{(i)}$ becomes closer to the true output $y^{(i)}$. To achieve this, we compute the partial derivatives of the loss function $\mathcal{L}$ with respect to each parameter vector θ:

$$\nabla_\theta \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}) = \left[\frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})}{\partial w_1}, \ldots, \frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})}{\partial w_P} \right].$$

Since both the NN and the loss function are compositions of differentiable functions, these derivatives can be efficiently computed using the chain rule. Each component of the gradient $\frac{\partial \mathcal{L}(\ldots)}{\partial w_k}$ $\forall k \in [1, \ldots, P]$ indicates how a small increase in the corresponding parameter w_k affects the loss. More precisely, for a given parameter, a positive value in the corresponding gradient component indicates that increasing this parameter will locally increase the loss. Conversely, a negative value indicates that increasing it will locally decrease the loss.

Now that we have information on how changes in the parameters affect the loss, we can aim to minimize it. To do so, the parameters are updated in the direction opposite to the gradient. Indeed, we move against the gradient because it indicates the direction in which the loss increases, whereas our goal is to reduce it. This leads us to the gradient descent update rule:

$$\theta^{(n+1)} = \theta^{(n)} - \eta \nabla_{\theta^{(n)}} \mathcal{L}(\hat{f}_{\theta^{(n)}}(x^{(i)}), y^{(i)}),$$

where $\eta > 0$ is the learning rate, which controls the step size. This parameter must remain small, as the gradient only provides reliable information in a local neighborhood of the current parameters. As a result, a single step is not sufficient to significantly reduce the loss, so the process is iterated many times. Producing a sequence of parameter vectors $\theta_1, \theta_2, \ldots$ that progressively improve the model. In practice, training is stopped when successive updates no longer produce a significant decrease in the loss.

It is through this simple set of rules: random initialization, gradient computation, and iterative parameter updates, we are able to train highly complex NNs. Of course, when using gradient descent to optimize the parameters of a non-convex function such as a NN, there is no theoretical guarantee of convergence to a global, or even a local, minimum. However, in practice, this method often yields satisfactory results for the task. This success is often attributed to the high expressive power of NNs, even though a complete theoretical understanding of this phenomenon remains an open question.

1.5 Evaluation

Now that we have described how to train a model, we must turn to the question of how to evaluate it. In the supervised learning setting, this evaluation relies on two aspects:

- its performance on the data it was trained on,
- its performance on data it was not trained on.

The first aspect measures how well the loss minimization process has succeeded. The second measures how well the model generalizes. Indeed, the dataset $\mathcal{D}$ represents a small subset of all possible inputs. To estimate how the model will perform in general, it is important to evaluate how it performs on data it has not seen during training.

In order to quantify these two aspects, the dataset is split into two subsets:

- $\mathcal{D}_{\text{train}}$ is the set on which the model minimizes its loss during the training.
- $\mathcal{D}_{\text{test}}$ is kept aside during training in order to evaluate the model's ability to generalize.

When training is completed, the first step is to verify that the model performs well on the data it was trained on. To do so, we study the average loss on the training set, denoted $\bar{\ell}_{\text{train}}$:

$$\bar{\ell}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{train}}} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})$$

If $\bar{\ell}_{\text{train}}$ is not sufficiently low, this means that the model has not been able to learn the task. It is not necessary to go further in the analysis of the model's quality. Before restarting the training process, various design choices are then adjusted. Based on what we have discussed so far in this chapter, these include modifications to the number of layers, the number of neurons per layer, activation functions, the learning rate, or the initialization strategy. However, in practice, there are many other possible variations. There is no universal rule for choosing the appropriate value for each of them. They are therefore chosen empirically, based on models that perform well on similar tasks, combined with intuition. Their selection is often costly and remains a challenging problem in DL.

Assuming now that the model achieves satisfactory performance on the training dataset $\mathcal{D}_{\text{train}}$. We must now study its ability to generalize. To do so,

we compute the corresponding average loss on $\mathcal{D}_{\text{test}}$:

$$\bar{\ell}_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{test}}} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})$$

If $\bar{\ell}_{\text{test}} \gg \bar{\ell}_{\text{train}}$, the model fails to generalize. This situation most often arises due to a phenomenon known as overfitting. In such cases, the model does not learn general patterns but instead memorizes the training examples. This typically occurs when the number of parameters is too large relative to the amount of training data. The simplest remedy is therefore to reduce the number of parameters in the model. However, it is still necessary to ensure that the model remains sufficiently expressive to perform well on the data.

If $\bar{\ell}_{\text{test}} \approx \bar{\ell}_{\text{train}}$, this indicates that the model generalizes well beyond the training data. The model can therefore be used with confidence for the task on which it was trained.

Bibliography

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901, 2020.

[2] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In Proceedings of the 10th ACM conference on recommender systems, pages 191–198, 2016.

[3] Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. Deep learning, volume 1. MIT press Cambridge, 2016.

[4] Martin Hilbert and Priscila López. The worlds technological capacity to store, communicate, and compute information. science, 332(6025):60–65, 2011.

[5] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. Neural networks, 2(5):359–366, 1989.

[6] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. nature, 596(7873):583–589, 2021.

[7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems, 25, 2012.

[8] Warren S McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. The bulletin of mathematical biophysics, 5(4):115–133, 1943.

[9] Marvin Minsky and Seymour Papert. An introduction to computational geometry. Cambridge tiass., HIT, 479(480):104, 1969.

[10] Tom M. Mitchell. Machine Learning. McGraw-Hill, New York, 1997.

[11] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. Psychological review, 65(6):386, 1958.

[12] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. nature, 323(6088):533–536, 1986.

[13] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815, 2017.